Course: Cloud Computing Lab **Course Code**: CS402

Sem: VII Sem CSE

DevOps Experiment using Jenkins on Windows but without using Docker

1. Install Java

Jenkins requires Java.

1. Download and install **Java 21 (LTS)**.
2. After installation, open **Command Prompt**.
3. Check Java:

```
java -version
```

You should see something similar to:

```
java version "21.x.x"
```

If `java` is not recognized, Java's PATH needs to be configured.

---

2. Download Jenkins for Windows

Go to the official Jenkins website and download the **Windows installer (.msi)**.

Jenkins official website

Choose:

Download Jenkins → Windows → Installer

The installer will look something like:

```
Jenkins.msi
```

3. Install Jenkins

Double-click the downloaded .msi file.
During installation:

Step 1 — Setup

Click: **Next**

Step 2 — Installation folder

You can keep the default location. For example: C:\Program Files\Jenkins

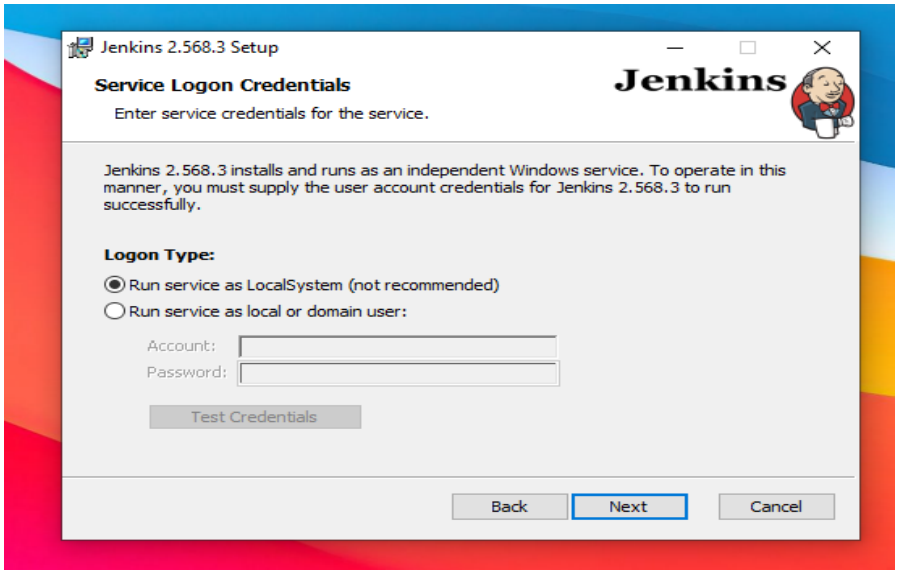
Click **Next**.

Step 3 — Service configuration

Jenkins can run as a **Windows Service**.

Select “Run service as LocalSystem” option:

Jenkins normally uses: 8080



For a basic college/lab setup, this is fine.

Take a screenshot here - 1

Step 4 — Port

So Jenkins will be accessible at:

http://localhost:8080

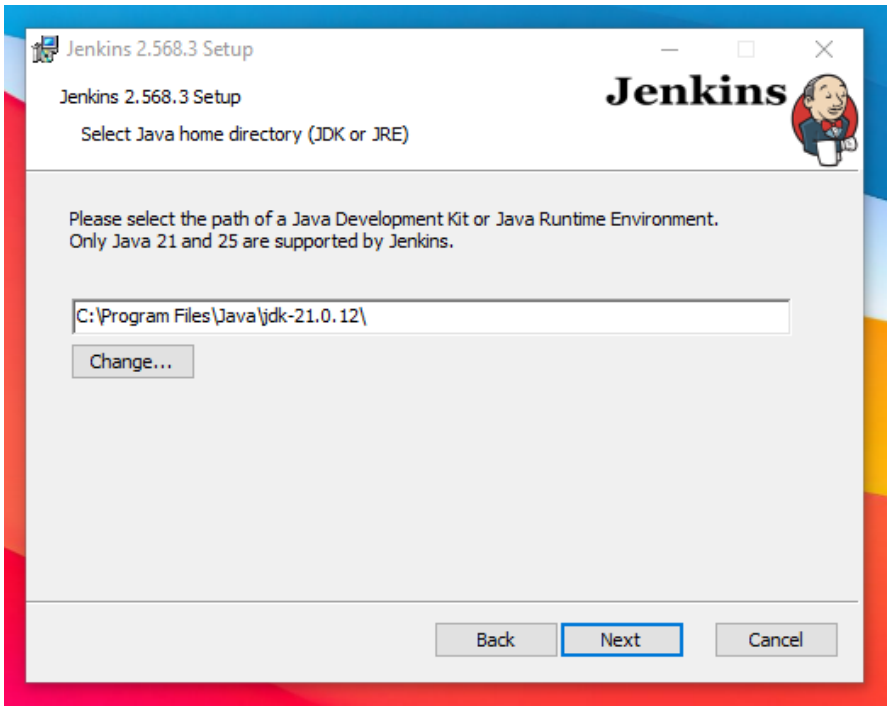
Keep 8080 unless another application is already using it in which case you can change the port number to 8081 or some other free port and test it using “Test port”

Step 5 — Java

The installer should detect your Java installation.

Make sure it detects Java 21.

If it does not auto detect Java 21 then manually change to path of Java 21 as shown below



Step 6 — Install

Click: **Install**

Wait for the installation to finish.

Take a screenshot here - 2

4. Start Jenkins

After installation, Jenkins normally starts automatically as a Windows service.

You can check it:

1. Press **Windows + R**
2. Type:

```
services.msc
```

3. Press Enter.
4. Find:

```
Jenkins
```

The status should be:

```
Running
```

If it is stopped:

Right-click → Start

---

5. Open Jenkins

Open Chrome/Edge and go to:

http://localhost:8080
(use the correct port number from Step 4)

You should see:

Unlock Jenkins

Jenkins will ask for an initial administrator password.

---

6. Find the Jenkins password

Open Command Prompt and run the following command:

```
type "C:\ProgramData\Jenkins\.jenkins\secrets\initialAdminPassword"
```

You will get a long password such as:

```
a8f3c9.....
```

Copy it.

Paste it into the Jenkins browser page.

If that path doesn't exist on your installation, the Jenkins service configuration may show a different JENKINS_HOME location.

7. Install Jenkins plugins

After entering the password, Jenkins gives you two choices:

Install suggested plugins

and

Select plugins to install

For your first setup, choose:

Install suggested plugins

Jenkins will install commonly required plugins.

Wait until installation finishes.

---

8. Create your Jenkins account

You will see:

Create First Admin User

Enter:

Username: admin
Password: ********
Full name: Your Name
Email: your-email@example.com

Then click:

Save and Continue

---

9. Jenkins is ready

You should now see the Jenkins dashboard.

Your Jenkins address is:

http://localhost:8080
(use the correct port number from Step 4)

You can use this address whenever Jenkins is running.

---

10. Create your first Jenkins job

From the Jenkins dashboard:

New Item

Enter a name, for example:

MyFirstProject

Select:

Freestyle project

Click:

OK

Take a screenshot here - 3

---

11. Add a simple test command

Scroll down to:

Build Steps

Click:

Add build step → Execute Windows batch command

Enter:

```
echo Hello from Jenkins
echo Jenkins is running successfully!
```

Click:

Save

---

12. Run the Jenkins job

On the project (i.e MyFirstProject) page, click:

Build Now

You will see:

Build #1

Click:

Build #1 → Console Output

You should see:

```
Hello from Jenkins
Jenkins is running successfully!
Finished: SUCCESS
```

🎉 This means Jenkins is working correctly.

---

13. Run a C program using Jenkins

Since you're working with C programs, you can also test Jenkins with a simple C program.

For example, create:

hello.c file in C:\JenkinsLab folder

```
#include <stdio.h>

int main()
{
   printf("Hello from C\n");
   return 0;
}
```

Install a C compiler such as **MinGW-w64** and make sure gcc works: (You can use visual studio or any other C compiler instead of MinGW-w64 to compile the C program)

- GCC compiler installed.
- GCC added to the Windows PATH.

gcc –version

Restart Jenkins: Windows + R

services.msc

Then in Jenkins:

Create a New Job

Click -> New Item

Enter the project name.

Select -> Freestyle Project

Click-> Ok

Configure the Project:

Build Steps → Execute Windows batch command

Enter Build Commands

Type the following commands.

cd /d C:\JenkinsLab

gcc hello.c -o hello.exe

hello.exe

Click->Save

Build the Project:

Click->Build Now

View Console Output:

The Jenkins console should show:

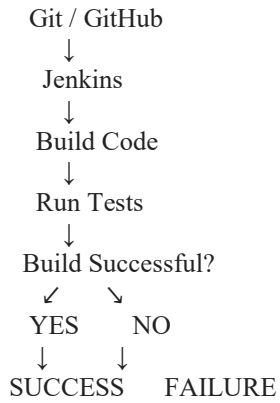
```
Hello from C
Finished: SUCCESS
```

Take a screenshot here - 4

Basic Jenkins workflow

The overall process is:

Your Code
↓



For example:

Developer
↓
Push code to GitHub
↓
Jenkins detects/builds it
↓
Download source code
↓
Compile
↓
Run test cases
↓
Generate result

Integrating Jenkins with GitHub Repository and Executing a C Program: (This is an optional step for learning/practice and not for submission)

Install Git

If you're going to use Jenkins with Git/GitHub, install Git for Windows.

Git official website

After installation, check:

```
git --version
```

Example:

```
git version 2.x.x
```

Prerequisites

1. **Java installed.**
2. **Jenkins installed and running.**
3. **Git installed.**
4. **GCC installed.**
5. **GitHub account created**

Step 1: Create a GitHub Repository

1. Login to GitHub.
2. Click New Repository.
3. Enter the repository name.
   Example: Jenkins_CC_Lab
4. Select Public.
5. Do not initialize with README, .gitignore, or License.
6. Click Create Repository.

Step 2: Create a Local Project

Create a folder.
C:\Jenkins_CC_Lab
cd Jenkins_CC_Lab

Create a file named hello.c

Paste the following code.

```
#include <stdio.h>

int main()
{
   printf("Hello from GitHub using Jenkins\n");
   return 0;
}
```

Save the file.

Step 3: Initialize Git Repository

If GitHub account is not connected to your local machine then:

git config --global user.name "Your Name"
git config --global user.email your-email@example.com
git config --list

and continue with below Else steps.

Else

Open Command Prompt inside the project folder.
Run
git init
git add .
git commit -m "Initial Commit"
git branch -M main
git remote add origin https://github.com/<username>/Jenkins_CC_Lab.git
git push -u origin main

After successful execution, the source code is uploaded to GitHub.

Step 4: Verify GitHub Repository

Open
https://github.com/<username>/Jenkins_CC_Lab
Verify that
hello.c
is available.

Step 5: Open Jenkins

Open the browser.
Go to
http://localhost:8080
use the correct port number that you specified while installing Jenkins.
Login to Jenkins.

Step 5: Create a New Jenkins Job

Click
New Item
Project Name
Ex: GitHub_CC or MySecondProject
Select
Freestyle Project
Click-> OK

Step 6: Configure Source Code Management

Select Git under “Source Code Management”
Repository URL
https://github.com/<username>/Jenkins_CC_Lab.git

Step 7: Configure Branch

Under

Branches to build
Enter
*/main
(if you see */master, change it to */main)

Step 8: Configure Build Step

Scroll to
Build Steps
Click
Add Build Step
Select
Execute Windows Batch Command
Enter
gcc hello.c -o hello.exe

hello.exe

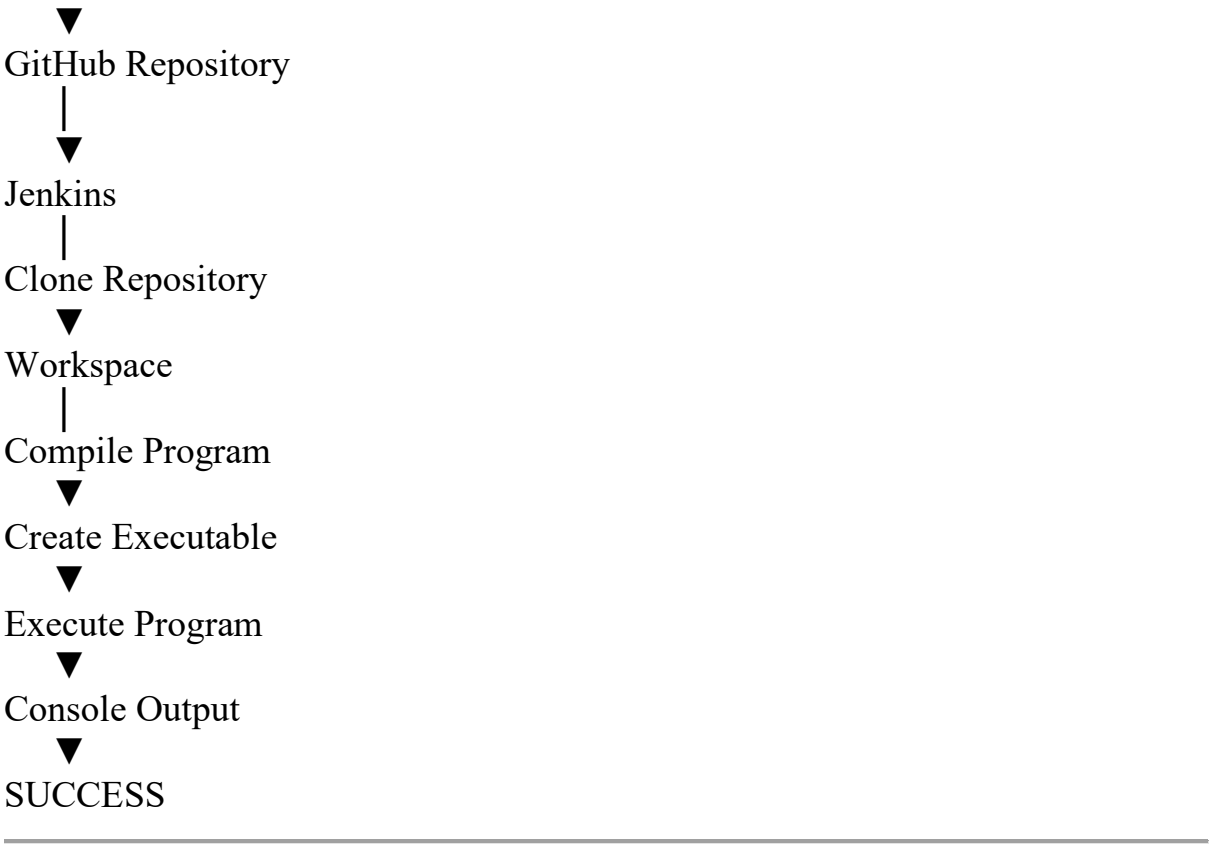
Click->Save.
Build the Project

Step 9: View Console Output

Click
Build History
↓
Select latest build.
↓
Click
Console Output

Workflow Diagram:

Developer
  |
Write Source Code
  |
  ▼
Local Repository
  |
git add
  |
git commit
  |
git push


GitHub Repository
Jenkins
Clone Repository
Workspace
Compile Program
Create Executable
Execute Program
Console Output
SUCCESS